

INDEX

INDEX

Name : Sayyed Mashhuda Begam Abde Muncif
STD. : BVOC. SD. 24 Div. : _____ Roll No. : 57
Sub. : SE
School / College : _____

[illegible]

Assignment - 5

1) Describe testing objectives and principles.

→ Testing objectives

1) Find Defects (Errors)

The main objective is to identify bugs and errors in the software before delivery.

2) Ensure Quality

Testing checks whether the software meets required quality standards.

3) Verify Requirements

It ensures the software works according to user requirements and specifications.

4) Improve Reliability

Testing helps make the system stable and reliable for users.

5) Increase user satisfaction

By reducing errors, testing ensures a better user experience.

Testing principles:

1) Testing shows presence of defects

Testing can prove that defects exist, but it cannot prove that the software is completely error-free.

2) Exhaustive testing is not possible

It is not possible to test all inputs and conditions, so testing focuses on important areas.

3) Early testing

Testing should start early in the development process to find defects sooner.

4) Defect clustering

Most defects are found in a small number of modules.

5) Pestieide Paradox

Repeating the same test cases again and again will not find new bugs; test should be updated.

6) Testing is context-dependent

Different projects require different testing approaches.

7) Absence of errors fallacy

Even if no errors are found, the software may still not meet user needs.

Testing objectives and principles help in improving software quality and ensuring the system works correctly and efficiently.

2) Explain the Psychology of testing.

→ The Psychology of testing refers to the mindset and attitude of testers and developers while testing software. It focuses on how people think and behave during testing to find more defects.

Key Points of Psychology of Testing:

1) Objective Mindset

Testing should be done with an unbiased view. The tester should try to find errors, not prove the software is correct.

2) Independent Testing

Testing is more effective when done by someone other than the developer, because developers may overlook their own mistakes.

3) Positive Attitude

Finding bugs should be seen as a positive activity that improves quality, not as criticism.

4) Avoid Ego Issues

Developers should accept errors without feeling bad. Testers and developers must work as a team.

5) Clear Communication

Bugs should be reported clearly and politely to avoid misunderstanding or conflict.

6) User Perspective

Testers should think like end users to identify real-world problems.

Good Psychology in testing improves teamwork, reduces conflict, and helps in finding more defects, leading to better software quality.

3) Explain various testing strategies for conventional software.

→ testing strategies conventional software

1) Unit Testing

2) Integration Testing

3) Validation Testing

4) System Testing

5) Alpha Testing

6) Beta Testing

1) Unit Testing

Each small Part (Module / Function) of the software is tested separately to check if it work correctly.

2) Integration Testing

Different modules are combined and tested together to ensure they interact properly.

3) Validation Testing

The complete software is tested to check whether it meets user requirements and specifications.

4) System Testing

The entire system is tested as a whole in a real environment. It checks performance, security, and overall behavior.

5) Alpha Testing

Testing is done by developers or internal users at the developer's site before releasing the software.

6) Beta Testing

Testing is done by actual users in a real environment before the final release to get feedback.

These strategies help in testing software step-by-step, from small units to the complete system, ensuring quality and reliability.

4) Explain black box testing with an example.

→ The black box testing is also called "Behavioural Testing".

Black box testing is a testing method where the tester checks the functionality of software without knowing the internal code or logic.

The tester only provides inputs and verifies the outputs.

- Focuses on what the system does, not how it works.
- The black box testing is not an alternative to white box testing.
- Useful for finding missing functions, incorrect outputs, and interface errors.

Example:

Suppose there is a login system:

- Input: Username = "admin", Password = "1234"
- Expected output: login successful

In black box testing, the tester will:

- Enter valid and invalid usernames / Passwords.
- check if login works correctly or shows error messages.

The tester does not know how the code checks the password internally, only the result.

Black box testing helps ensure that the software behaves correctly according to user requirements without focusing on internal implementation.

Common Techniques:

1) Equivalence Partitioning

2) Boundary Value Analysis

3) Decision Table Testing.

- No knowledge of programming or internal structure is required.
- Tests input - output behaviour of the system
- can be done by tester or even end user.

White box testing is also called "glass box testing".

5) Explain white box testing with an example.

→ White box testing is a testing method where the tester checks the internal structure, logic, and code of the software. The tester must have knowledge of programming and examines how the code works.

- Focuses on how the system works internally.
- Requires knowledge of code and logic.
- Tests conditions, loops, and paths in the program.
- Helps in finding logical errors and hidden bugs.
- Mostly done by developers.
- Used mainly in unit testing.

Example:

Suppose a program checks if a number is even or odd:

if (num % 2 == 0)

system.out.println ("Even");

else

system.out.println ("odd");

In white box testing, the tester will:

- check both condition (even and odd cases)
- Ensure each Path (if and else) is executed.
- verify the logic is correct.

White box testing ensures that the internal code works correctly and all possible paths are tested properly.

- White box testing Ensures proper flow of data inside the program.
- White box testing Helps in early bug detection during development.
- White box testing is not an alternative to a black box testing.

6) Write the short note on:

a) Regression Testing.

b) Validation Testing.

→ Regression testing is a type of testing that performed after modifications in software, such as bug fixes, update, or adding new features, to ensure that the existing functionality is not affected and work correctly.

When developers change code, there is a risk that previously working parts may break. Regression testing helps to detect such issues.

It involves re-running previously executed test cases to check that new changes have not introduced new errors.

Key point

- Done after every change or update in the software.
- Ensures no side effects on existing functionality.
- Uses previous test cases.
- Can be manual or automated.

- Important in maintenance phase of software.

Regression testing helps maintain software stability and ensures that updates do not break existing features.

by Validation Testing

Validation testing is performed to ensure that the final software product meets user requirement and expectations. It checks whether the system does what the user actually needs in real-world conditions.

This testing focuses on the overall behavior of the system, not on internal code or logic.

It is usually carried out after the development and integration phase are completed, just before the software is delivered to users.

Validation testing includes activities like system testing and acceptance testing, where the complete system is tested as a whole.

Real users or clients may also be involved to verify that the software is suitable for practical use.

Key Points:

- Ensures software meets user requirement.
- Ensures software is ready ~~to~~ for real-world use.
- Done at the final stage of testing.
- Focuses on functionality and user satisfaction.
- Includes activities like system testing and acceptance testing.
- May involve real user or client.

Validation testing ensures that right product has been built and that it satisfies user expectations, making the software ready for release.

7) Explain system testing and recovery testing.

→ System Testing

System testing is the process of testing the complete and Full integrated software system as a whole. It is performed after integration testing and checks whether the system meets all Functional and non-Functional requirements specified by the user.

The main goal is to verify that the entire system works correctly in an environment that is similar to real-world usage.

In system testing, different types of testing such as performance testing, security testing, usability testing, and stress testing may be included.

It ensures that all components work together properly and the system behaves as expected from start to end.

Key points:

- Tests the entire system as a whole
- validates both Functional and non-Functional requirements.

- Performed in a realistic environment.
- Includes multiple testing types (performance, security, etc.)
- Done before acceptance testing

system testing ensures that the complete software work correctly.

Recovery Testing

Recovery testing is performed to check how well the system can recover from unexpected failures such as system crashes, hardware faults, power failures, or network issues.

It ensures that the system can restore data, restart operations, and continue working without major loss.

This testing involves intentionally causing failures and observing how quickly and effectively the system recovers.

It also checks the backup and restore mechanisms to ensure data safety and system reliability.

Key points:

- Tests system ability to recover from failures
- Includes scenarios like crashes, power failure, data loss.
- Verifies backup and restore processes
- Ensures minimum data loss and quick recovery

• Improves system reliability and stability.

Recovery testing ensures that it can handle failures and recover efficiently, making the system more reliable.

Q) What is Agility? Describe its principle.

→ Agility is the ability of a software development process to quickly respond to changes and deliver working software in a flexible and efficient way. It focuses on customer satisfaction, continuous improvement, and fast delivery.

Principles of Agility:

- 1) customer satisfaction
- 2) Welcome change
- 3) Frequent delivery
- 4) collaboration
- 5) Face-to-face communication
- 6) Working software as measure
- 7) sustainable development
- 8) Focus on Quality
- 9) simplicity
- 10) continuous improvement

1) customer satisfaction :- deliver useful software early and continuously.

2) Welcome change :- Accept changing requirements at any stage.

3) Frequent delivery :- deliver working software in small parts regularly.

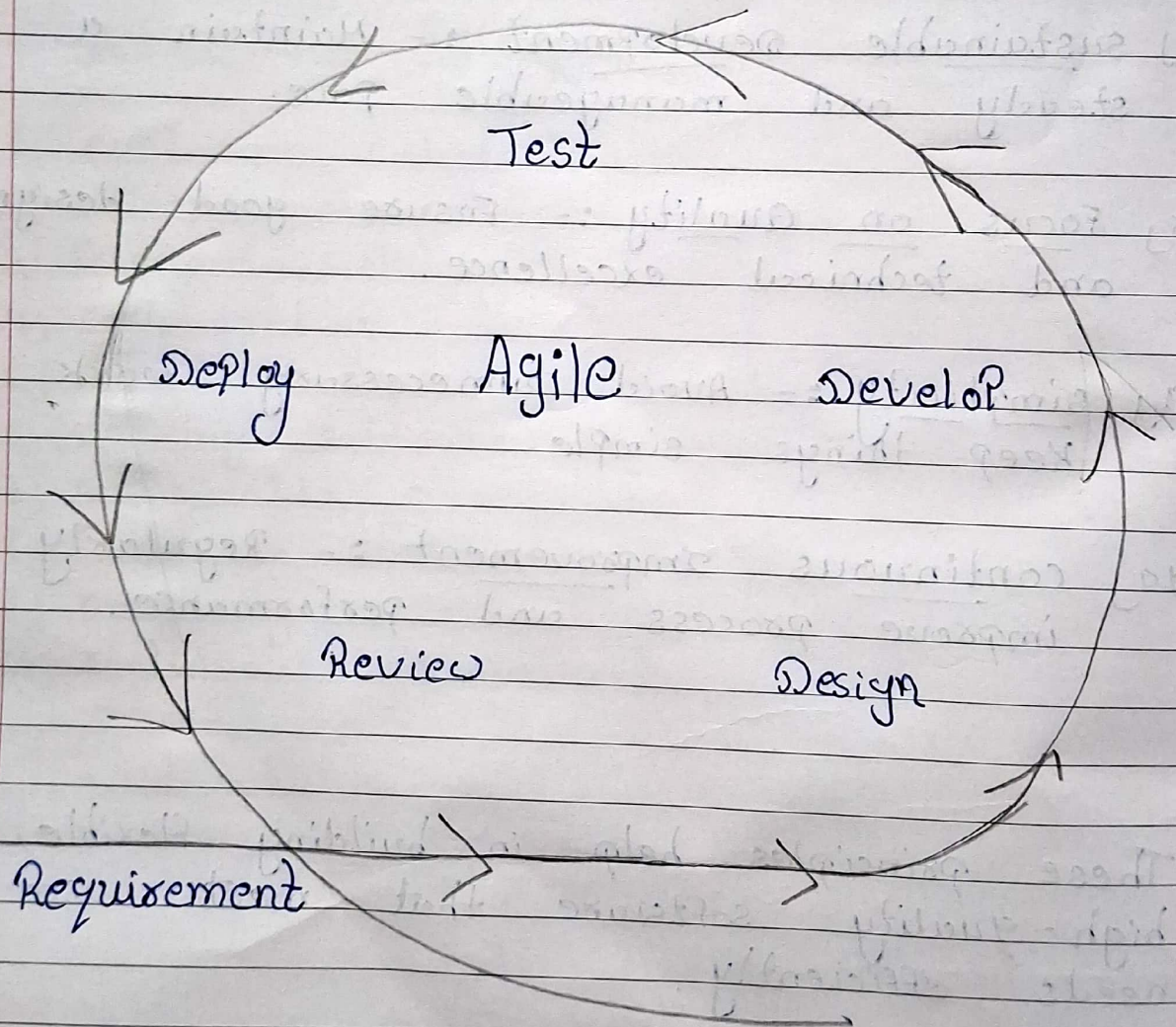
- 4) collaboration :- Developers and customers work together closely.
- 5) Face-to-Face communication :- Direct communication is most effective.
- 6) Working software as Measure :- Progress is measured by working software.
- 7) sustainable development :- Maintain a steady and manageable pace.
- 8) Focus on Quality :- Ensure good design and technical excellence.
- 9) simplicity :- Avoid unnecessary work and keep things simple.
- 10) continuous improvement :- Regularly improve process and performance.

These principles help in building flexible, high-quality software that meets user needs efficiently.

Q) Discuss agile process models.

→ Agile software development is a software development approach in which work is done in small parts (iterations), with continuous feedback and improvement.

Agile process Model diagram



1) Requishment

Requishment are collected in small parts. These can change anytime based on customer needs. They are usually written as simple user stories.

2) Design

A basic and quick design is prepared only for the current features avoids heavy documentation and focuses on simplicity.

3) Develop

Developers write code for selected features in the iteration. The goal is to create a working product quickly.

4) Test

Testing is done continuously along with development. Bugs are identified and fixed early to maintain quality.

5) Review

The developed product is reviewed by customers or stakeholders. Feedback is taken to improve the next version.

6) Deploy

The tested and reviewed product is released to users. This may be a complete or partial version of the software.